

SQL PROJECT

1. Objective:

Design and implement a relational database for an E-Commerce system using SQL.

Create related tables using primary and foreign keys, insert sample data, and write SQL queries of varying difficulty levels.

2. Create Database:

```
CREATE DATABASE ECommerceDB;
```

4. Data Insertion Requirement:

Customers:

```
INSERT INTO Customers VALUES
```

```
(101, 'Lakshmi', 'Menon', 'lakm@gmail', '9876543210', 'Bangalore', '2024-11-10'),  
(102, 'Ravi', 'Kumar', 'ravi@gmail', '9876543211', 'Mumbai', '2024-11-15'),  
(103, 'Sneha', 'Reddy', 'sneh@gmail', '9876543212', 'Hyderabad', '2024-12-01'),  
(104, 'Arjun', 'Patel', 'apa@gmail', '9876543213', 'Delhi', '2024-12-05'),  
(105, 'Divya', 'Sharma', 'dsha@gmail', '9876543214', 'Pune', '2025-01-02'),  
(106, 'Manoj', 'Singh', 'mano@gmail', '9876543215', 'Kolkata', '2025-01-10'),  
(107, 'Kavya', 'Das', 'das@gmail', '9876543216', 'Chennai', '2025-02-20'),  
(108, 'Anil', 'Rao', 'anil@gmail', '9876543217', 'Mumbai', '2025-03-01'),  
(109, 'Priya', 'Verma', 'pver@gmail', '9876543218', 'Goa', '2025-04-05'),  
(110, 'Rahul', 'Iyer', 'iyer@gmail', '9876543219', 'Chennai', '2025-05-05');
```

Categories:

```
INSERT INTO Categories VALUES
```

```
(201, 'Electronics'),  
(202, 'Fashion'),
```

(203, 'Home Appliances'),

(204, 'Books'),

(205, 'Groceries');

Products:

INSERT INTO Products VALUES

(301, 'Mobile', 201, 45000.00, 30),

(302, 'Laptop', 201, 72000.00, 15),

(303, 'Shirt', 202, 900.00, 50),

(304, 'Shoes', 202, 2500.00, 40),

(305, 'RiceBag', 205, 850.00, 80),

(306, 'Microwave', 203, 16000.00, 10),

(307, 'Novel', 204, 500.00, 25),

(308, 'Fridge', 203, 28000.00, 12),

(309, 'Headphones', 201, 3500.00, 35),

(310, 'Tablet', 201, 29000.00, 20);

Orders:

INSERT INTO Orders VALUES

(401, 101, '2025-08-10', 90000.00, 'Delivered'),

(402, 102, '2025-09-05', 50000.00, 'Delivered'),

(403, 103, '2025-09-07', 1800.00, 'Delivered'),

(404, 104, '2025-09-15', 3400.00, 'Pending'),

(405, 105, '2025-09-25', 5600.00, 'Delivered'),

(406, 106, '2025-09-29', 28000.00, 'Delivered'),

(407, 107, '2025-10-05', 72000.00, 'Delivered'),

(408, 108, '2025-10-07', 2750.00, 'Cancelled'),

(409, 109, '2025-10-15', 900.00, 'Delivered'),

(410, 103, '2025-10-20', 7200.00, 'Pending');

OrderItems:

INSERT INTO OrderItems VALUES

(601, 401, 301, 1, 45000.00),

(602, 401, 302, 1, 45000.00),

(603, 402, 306, 2, 16000.00),

(604, 402, 304, 3, 2500.00),

(605, 403, 303, 2, 900.00),

(606, 404, 305, 4, 850.00),

(607, 405, 304, 2, 2500.00),

(608, 406, 308, 1, 28000.00),

(609, 407, 302, 1, 72000.00),

(610, 408, 303, 3, 900.00);

5. SQL Problem Statements:

5.1 Easy (3 Problems):

1. Display all customers who live in a specific city (e.g., 'Mumbai').

Solution:

```
SELECT *  
FROM Customers  
WHERE city = "Mumbai";
```

	customer_id	first_name	last_name	email	phone	city	registration_date
▶	102	Ravi	Kumar	ravi@gmail	9876543211	Mumbai	2024-11-15
	108	Anil	Rao	anil@gmail	9876543217	Mumbai	2025-03-01
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

2. List all products with a price greater than 1000.

Solution:

```

SELECT *
FROM Products
WHERE price > 1000;

```

	product_id	product_name	category_id	price	stock_quantity
▶	301	Mobile	201	45000.00	30
	302	Laptop	201	72000.00	15
	304	Shoes	202	2500.00	40
	306	Microwave	203	16000.00	10
	308	Fridge	203	28000.00	12
	309	Headphones	201	3500.00	35
	310	Tablet	201	29000.00	20
•	NULL	NULL	NULL	NULL	NULL

3. Show order_id, order_date, and total_amount of all orders sorted by most recent order first.

Solution:

```

SELECT
    order_id,
    order_date,
    total_amount
FROM Orders
ORDER BY order_date DESC;

```

	order_id	order_date	total_amount
▶	410	2025-10-20	7200.00
	409	2025-10-15	900.00
	408	2025-10-07	2750.00
	407	2025-10-05	72000.00
	406	2025-09-29	28000.00
	405	2025-09-25	5600.00
	404	2025-09-15	3400.00
	403	2025-09-07	1800.00
	402	2025-09-05	50000.00
	401	2025-08-10	90000.00
•	NULL	NULL	NULL

5.2 Medium (4 Problems):

4. Display customer full name and the total number of orders they have placed.

Solution:

```
SELECT
    CONCAT(cu.first_name, ' ', cu.last_name) AS Full_Name,
    COUNT(o.order_id) AS Total_Order
FROM Customers AS cu
JOIN Orders AS o
    ON cu.customer_id = o.customer_id
GROUP BY cu.customer_id
ORDER BY Total_Order DESC;
```

	Full_Name	Total_Order
▶	Sneha Reddy	2
	Lakshmi Menon	1
	Ravi Kumar	1
	Arjun Patel	1
	Divya Sharma	1
	Manoj Singh	1
	Kavya Das	1
	Anil Rao	1
	Priya Verma	1

5. List all product names along with their category name.

Solution:

```
SELECT
    p.product_name,
    c.category_name
FROM Products AS p
JOIN Categories AS c
```

```
ON p.category_id = c.category_id
ORDER BY p.product_name;
```

	product_name	category_name
►	Fridge	Home Appliances
	Headphones	Electronics
	Laptop	Electronics
	Microwave	Home Appliances
	Mobile	Electronics
	Novel	Books
	RiceBag	Groceries
	Shirt	Fashion
	Shoes	Fashion
	Tablet	Electronics

6. Find the total quantity sold for each product.

Solution:

```
SELECT
    p.product_name,
    SUM(o.quantity) AS Total_Quantity
FROM Products AS p
LEFT JOIN OrderItems AS o
    ON p.product_id = o.product_id
GROUP BY
    p.product_id,
    p.product_name
ORDER BY Total_Quantity DESC;
```

	product_name	Total_Quantity
▶	Shirt	5
	Shoes	5
	RiceBag	4
	Laptop	2
	Microwave	2
	Mobile	1
	Fridge	1
	Novel	NULL
	Headphones	NULL
	Tablet	NULL

7. Show the top 3 most expensive products.

Solution:

```
SELECT
    product_name,
    price
FROM Products
ORDER BY price DESC
LIMIT 3;
```

	product_name	price
▶	Laptop	72000.00
	Mobile	45000.00
	Tablet	29000.00

5.3 Hard (Nested Queries) (3 Problems):

8. Display customers who have never placed any order. (Use a nested query with NOT IN or NOT EXISTS):

Solution:

```
SELECT
    CONCAT(first_name, ' ', last_name) AS Full_Name
FROM Customers
```

```
WHERE customer_id NOT IN (
    SELECT customer_id
    FROM Orders
);
```

	Full_Name
▶	Rahul Iyer

9. Find products that have never been ordered. (Use a subquery on the Orders or OrderItems table)

Solution:

```
SELECT
    product_name,
    price
FROM Products
WHERE product_id NOT IN (
    SELECT DISTINCT product_id
    FROM OrderItems
);
```

	product_name	price
▶	Novel	500.00
	Headphones	3500.00
	Tablet	29000.00

10. Display the name of the customer who has spent the highest total amount across all their orders. (Use a subquery with SUM and MAX):

Solution:


```

Full_Name      SELECT CONCAT(c.first_name, ' ', c.last_name) AS
Total_Spent    FROM Customers c
                JOIN (
                    SELECT customer_id, SUM(total_amount) AS
                    FROM Orders
                    GROUP BY customer_id
                ) AS o_sum
                ON c.customer_id = o_sum.customer_id
WHERE o_sum.total_spent = (
    SELECT MAX(total_spent)
    FROM (
        SELECT SUM(total_amount) AS Total_Spent
        FROM Orders
        GROUP BY customer_id
    ) AS Subquery
);

```

	Full_Name
▶	Lakshmi Menon

